



수학 시뮬레이터 제작 보고서

수행평가

《시뮬레이션 파일 저장하는 방법》

- ① 최종 시뮬레이터의 html 코드를 전체 복사한다.
- ② 윈도우 메모장을 켜 후 위의 코드를 붙여넣기 한다.
- ③ 메모장에서 파일 - 다른 이름으로 저장 - 각각 본인이름1.html, 본인이름2.html으로 저장
(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

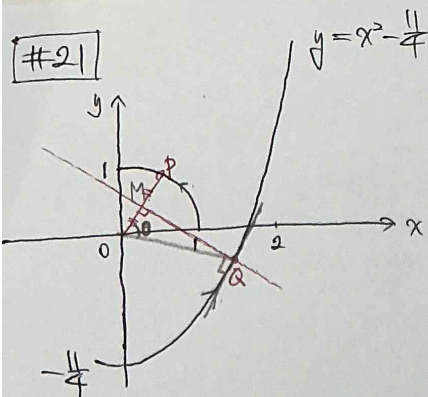
[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

점 P 는 원 $x^2 + y^2 = 1$ 위의 점 $(1, 0)$ 으로부터 시계 반대 방향으로 돌아 점 $(0, 1)$ 까지 움직인다. 원점을 O 라 하고, 선분 OP 의 수직이등분선과 포물선의 오른쪽 반부분 $y = x^2 - \frac{11}{4} (x \geq 0)$ 인 부분과의 교점을 Q 라 하자.

- (1) 점 Q 의 좌표를 $\left(x, x^2 - \frac{11}{4}\right)$ 이라 나타낼 때, x 값의 범위를 구하라.
- (2) 이등변삼각형 OPQ 의 넓이가 최소가 될 때, 점 P 와 점 Q 의 좌표를 구하라.
(+ α #21)

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)



1) Q_x 는 증가함수

$P(1, 0)$ 에서 $Q(\frac{1}{2}, -\frac{1}{2})$

$P(0, 1)$ 에서 $Q(\frac{\sqrt{5}}{2}, \frac{1}{2})$

$$\therefore \frac{1}{2} \leq x \leq \frac{\sqrt{5}}{2}$$

2) $[\triangle OPQ] = \overline{OP} \times \overline{MQ} \times \frac{1}{2} \quad \angle POQ = \theta$

$$= \frac{1}{2} \overline{OQ} \sin \theta$$

$$= \frac{1}{2} \sqrt{(\overline{OQ})^2 - 1}$$

$$\cos \theta = \frac{1}{\overline{OQ}} \quad \therefore \sin \theta = \sqrt{1 - \left(\frac{1}{\overline{OQ}}\right)^2}$$

$\therefore$ 넓이가 최소 $\Leftrightarrow \overline{OQ}$ 가 최소

$$\overline{OQ}_{\min} : \sqrt{x^2 + \left(x^2 - \frac{1}{4}\right)^2} \text{가 최소}$$

$$\rightarrow x^4 - \frac{9}{2}x^2 + \frac{121}{16} = f(x)$$

$$f'(x) = 4x^3 - 9x = 0 \quad \Leftrightarrow x(4x^2 - 9) = 0 \dots$$

$$\therefore x = \frac{3}{2} \text{에서 최소} \dots$$

$$\cos \theta = \frac{1}{2\sqrt{\frac{10}{4}}} = \frac{1}{\sqrt{10}}$$

$$\sin \theta = \frac{3}{\sqrt{10}}$$

+0만큼 Q 를 회전!

$$Q\left(\frac{3}{2}, -\frac{1}{2}\right)$$

$$\therefore \begin{pmatrix} \frac{1}{\sqrt{10}} & -\frac{3}{\sqrt{10}} \\ \frac{3}{\sqrt{10}} & \frac{1}{\sqrt{10}} \end{pmatrix} \begin{pmatrix} \frac{3}{2} \\ -\frac{1}{2} \end{pmatrix} = \left(\frac{1}{2\sqrt{10}}(3+3), \frac{1}{2\sqrt{10}}(9-1) \right)$$

$$= \frac{1}{\sqrt{10}}(6, 8) \rightarrow \text{단위벡터} P\left(\frac{3}{5}, \frac{4}{5}\right)$$

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제는 점 P 가 정의역에서 원 위를 따라 이동하면 선분 OP 의 수직이등분선도 계속 회전하고, 그에 따라 점 Q 역시 포물선 $y = x^2 - \frac{11}{4}$ 위에서 연속적으로 움직이는 문제다. 그렇기 때문에 어느 한 점만 잡아서 그래프에 도시하고 문제를 풀어나갈 수 없어서, 점을 움직여가며 어떤 점에서 최소인지 알아내는 시뮬레이션이 필요하다고 생각했다.

먼저, 문제 상황을 정확하게 이해할 필요가 있었다. 점 P 의 위치 변화에 따라 수직이등분선과 점 Q 가 어떻게 변하는 지, 식만으로는 직관적으로 이해하기 어려웠다. 시뮬레이터를 통해 점 P 가 움직이는 모습을 보면서 Q 도 특정 범위에서 움직인다는 사실과 전체적인 구조를 시각적으로 이해할 수 있었다.

다음으로, 계산 결과를 검토하기 위해 필요했다. 소문항 (1)에서 구한 $\frac{1}{2} \leq x \leq \frac{\sqrt{13}}{2}$ 라는 범위와, (2)에서 얻은 $Q\left(\frac{3}{2}, -\frac{1}{2}\right)$ 가 실제 도형의 상황과 일치하는지 처음에는 확신하기 어려웠다. 하지만 시뮬레이션으로 점들의 움직임을 확인하면서 계산 결과가 문제 조건과 정확히 대응한다는 점을 검산할 수 있었다.

마지막으로, 풀이 방향에 대한 힌트를 얻기 위해 필요했다. 시뮬레이션으로 점 Q 를 움직여보니, 점 Q 가 원점 O 에 가장 가까워질 때 삼각형 OPQ 의 넓이도 최소가 된다는 사실을 직관적으로 확인할 수 있었다. 이를 통해 넓이 문제를 OQ 의 최솟값 문제로 바꾸어 접근해도 된다는 아이디어를 얻을 수 있었다.

4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

다음 미적분 문제를 이해하고 풀이를 보조할 수 있는 HTML 기반 시뮬레이터를 만들어줘.

점 P 는 원 $x^2 + y^2 = 1$ 위의 점 $(1, 0)$ 에서 출발하여 시계 반대 방향으로 움직여 점 $(0, 1)$ 까지 이동한다. 원점을 O 라 하고, 선분 OP 의 수직이등분선과 포물선 $y = x^2 - \frac{11}{4} (x \geq 0)$ 의 교점을 Q 라 한다.

시뮬레이터에는 다음 기능들이 들어가야 해

좌표평면에 원 $x^2 + y^2 = 1$ 의 제1사분면 호, 포물선 $y = x^2 - \frac{11}{4}$ 의 오른쪽 부분, 점 O , 점 P , 점 Q 를 정확히 표시해줘. 점 P 는 원 위에서 $(1, 0)$ 부터 $(0, 1)$ 까지 움직이도록 하고, 슬라이더를 이용해 사용자가 직접 위치를 조절할 수 있게 해줘. 자동 재생 버튼도 만들어서 점 P 가 연속적으로 움직이게 해줘.

점 P 가 움직일 때마다 선분 OP 와 그 수직이등분선을 함께 표시해 줘. 수직이등분선은 점 Q 를 지나야 하며, 점 Q 가 이 직선과 포물선의 교점으로 결정된다는 것을 시각적으로 알 수 있게 해줘.

점 Q 의 좌표를 실시간으로 화면에 표시해줘. 특히 Q 의 좌표가 $\left(x, x^2 - \frac{11}{4}\right)$ 꼴이라는 점이 잘 보이도록 x 값과 y 값을 따로 표시해줘.

(1)번 풀이를 검토할 수 있도록 점 Q 가 움직이는 구간을 포물선 위에 강조해서 표시해줘. 가능하면 양 끝점

$\left(\frac{1}{2}, -\frac{5}{2}\right)$ 과 $\left(\frac{\sqrt{13}}{2}, \frac{1}{2}\right)$ 도 표시해줘. (2)번 풀이를 돕기 위해 삼각형 OPQ 를 반투명하게 표시하고, 현재 삼각형의 넓이와 거리 OQ 를 실시간으로 계산해줘.

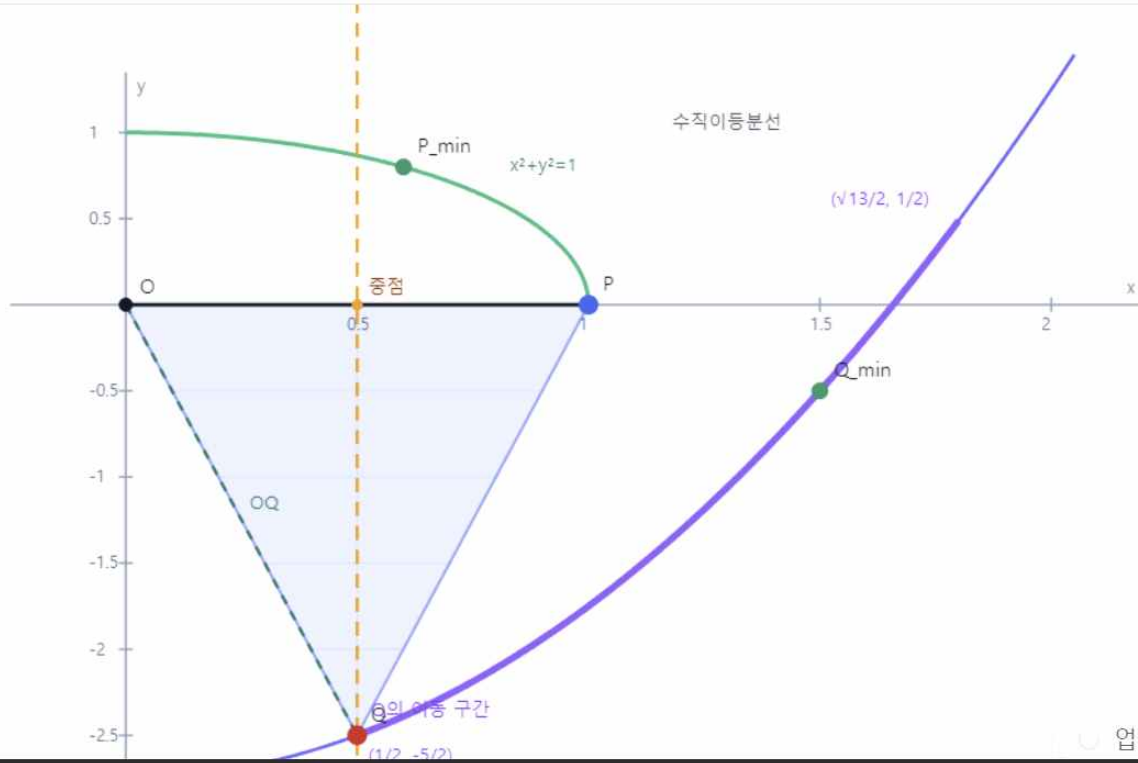
삼각형 OPQ 의 넓이가 최소가 되는 순간을 자동으로 찾거나, 그 지점을 특별한 색으로 표시해 줘. 이때 점 Q 가 (1)에서 구한 구간 안에 잘 들어오는 것을 화면에 나타내줘. 또 단순히 정답만 보여주는 것이 아니라, “삼각형 OPQ 의 넓이 최소는 OQ 의 최소와 연결된다”는 풀이 아이디어를 사용자가 이해할 수 있도록, OQ 길이가 작아질수록 넓이도 작아지는 모습을 시각적으로 보여줘.

디자인은 보고서에 캡처해서 넣기 좋도록 깔끔하게 만들어줘. 왼쪽에는 그래프, 오른쪽에는 현재 좌표, 거리, 넓이, 최소 지점 정보를 표시하는 패널을 배치해 줘. 외부 라이브러리 없이 하나의 HTML 파일 안에서 실행되도록 작성해 줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

좌표평면

원, 포물선, 수직이등분선, 삼각형 $\triangle OPQ$ 를 함께 표시하고, 점 P 를 움직이면 점 Q 와 넓이가 실시간으로 바뀐다.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

최초 시뮬레이터는 점 P 가 원 위를 움직일 때 수직이등분선과 점 Q 가 함께 변하는 모습을 보여 주었지만, 문제 풀이에 직접적으로 필요한 정보를 충분히 제공하지는 못했다. 먼저 좌표평면에서 x 축과 y 축의 눈금 비율이 정확히 같지 않아 원, 포물선, 수직이등분선의 관계가 실제와 다르게 보일 수 있었으므로, 두 축의 스케일을 동일하게 맞추어야 한다. 또한 점 Q 의 좌표가 소수 근삿값으로만 표시되면 풀이에서 얻은 참값과 비교하기 어렵기 때문에, 특

수한 위치에서는 $\frac{1}{2}, \frac{\sqrt{13}}{2}, \frac{3}{2}$ 처럼 정확한 값으로 표시되도록 수정해야 한다.

그리고 (2)번 풀이의 핵심은 OQ 의 길이가 최소가 될 때 삼각형 OPQ 의 넓이도 최소가 된다는 점이므로, 단순히 현재 넓이만 표시하는 것이 아니라 θ 가 변할 때 OQ 의 길이와 삼각형의 넓이가 어떻게 함께 변하는지를 그래프로 보여줄 필요가 있다. 이를 통해 두 값이 같은 지점에서 동시에 최소가 됨을 시각적으로 확인할 수 있도록 해야 한다.

마지막으로 최소 지점인 $Q\left(\frac{3}{2}, -\frac{1}{2}\right)$ 에서는 포물선의 접선이 선분 OQ 와 수직이 된다는 중요한 성질이 나타나기 때문에, 그 순간에만 접선과 수직 표시가 그래프에 나타나도록 수정해야 한다. 또한 풀이 힌트는 처음부터 모두 보이게 하기보다, 힌트 보기/끄기 버튼을 눌렀을 때만 나타나도록 하여 사용자가 먼저 직접 관찰한 뒤 필요한 경우에만 풀이 방향을 확인할 수 있게 해야 한다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

1번째 수정:

실행시키면 수식이 안 깨지고 나올 수 있도록 해. 렌더링 제대로 되게. 그리고 그래프를 그릴 때는 x, y 축 스케일을 똑같이 해서 그려. 그리고 Q_{min} 일 때 접선이 OQ 랑 수직인 것도 그때만 그래프에 그려지게 해.

양 극단값의 좌표는 그래프에 그려놓고, 옆에는 OQ 의 길이에 따른 삼각형 넓이 그래프도 동시에 그려.

2번째 수정:

OQ의 길이와 삼각형 넓이

이 부분을 좌표평면 옆에 도시하라고 같이

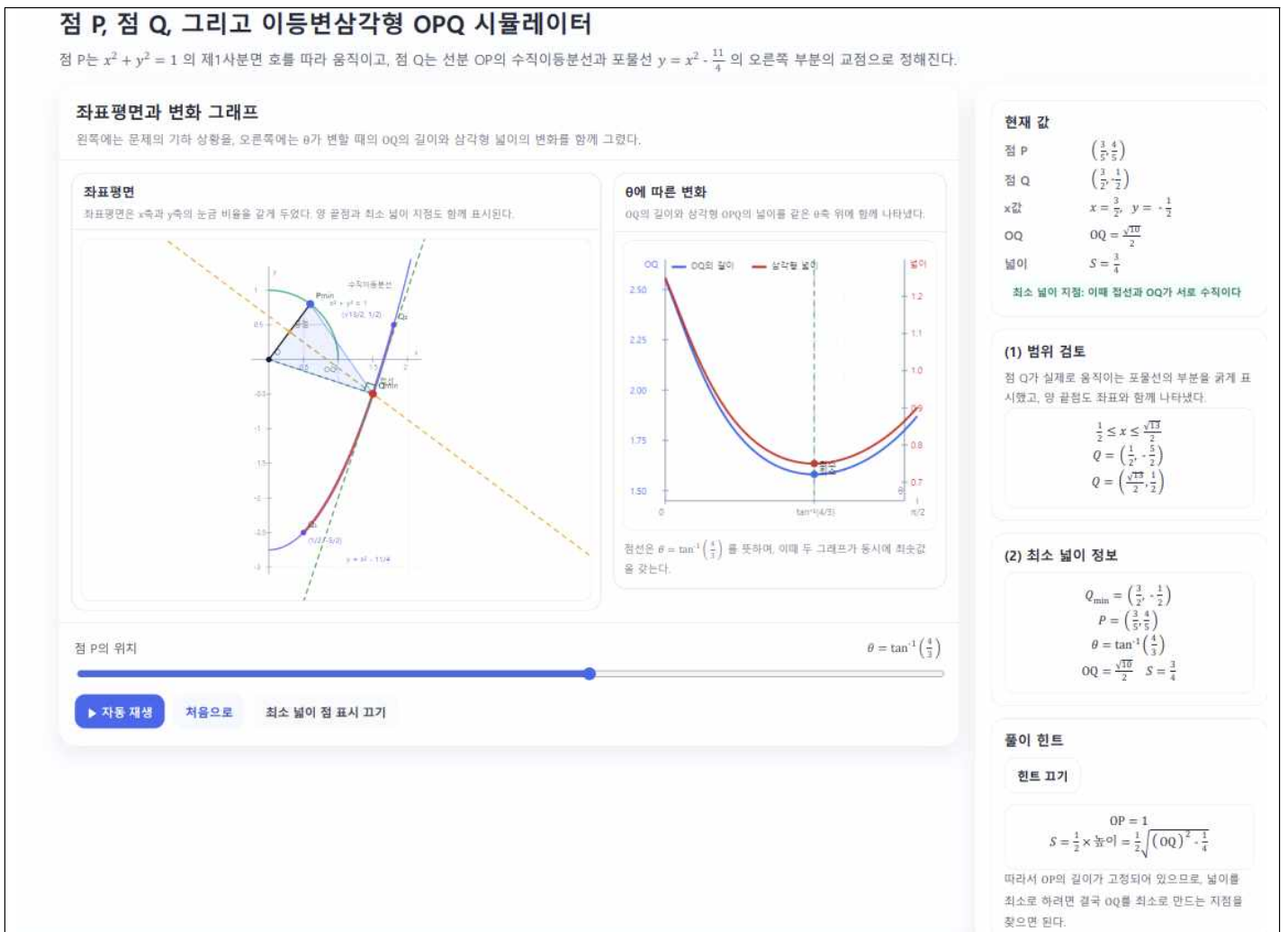
그리고 OQ의 길이와 삼각형의 넓이로 그래프를 그리지 말고, P가 변해가면서 시간(세타)에 따른 OQ의 길이와, 삼각형의 넓이 그래프를 같이 그려서 OQ 값이 최소일 때 삼각형 넓이도 최소임이 같이 드러나게 그려.

3번째 수정:

숫자로 나오는 값은 항상 참값으로 적어. 예를 들면 마지막에 $x = 1.8708$ 이런거는 루트 13/2로 적을 수 있잖아. 특수한 값은 그렇게 적고 마지막에 풀이 힌트는 옆에 힌트 보기/끄기 버튼을 눌러야 나오고, 한번 더 누르면 다시 꺼지는 방식으로 만들어.

그리고 수식을 유니코드 활용해서 적지 말고 좀 더 예쁜 폰트로 실제 수식처럼 써줘.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.



10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

화면의 슬라이더를 움직이면 점 P가 원 위를 따라 (1, 0)에서 (0, 1)까지 이동하고, 그에 따라 선분 OP의 수직이등분선과 포물선의 교점인 점 Q가 자동으로 함께 움직인다. 그래프에는 원, 포물선, 수직이등분선, 삼각형 OPQ가 동시에 표시되며, 오른쪽 패널에서는 점 P, 점 Q, OQ의 길이, 삼각형의 넓이를 확인할 수 있다. 자동 재생 버튼을 누르면 점 P가 연속적으로 움직이므로 전체 변화를 한 번에 관찰할 수 있다.

또한 좌표평면 옆의 그래프에서는 θ 가 변할 때 OQ의 길이와 삼각형의 넓이가 어떻게 함께 변하는지를 볼 수 있다. 사용자는 이 그래프를 통해 두 값이 같은 시점에서 최소가 된다는 사실을 확인할 수 있고, 최소 넓이 지점에 도달하면 그래프와 좌표평면에서 그 순간이 함께 강조된다. 필요하면 힌트 보기 버튼을 눌러 풀이의 핵심 아이디어도 확인할 수 있다.

11. 시뮬레이터가 자신이 의도한 대로 문제를 파악하거나 풀이 과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

이 시뮬레이터는 문제의 구조를 직관적으로 파악하는 데 큰 도움이 되었다. 점 P가 움직일 때 점 Q가 포물선의 어느 부분을 따라 이동하는지 바로 볼 수 있었고, $(\frac{1}{2}, -\frac{5}{2})$ 에서 $(\frac{\sqrt{13}}{2}, \frac{1}{2})$ 까지가 실제 이동 구간이라는 사실을 시각적으로 확인할 수 있었다. 또한 θ 에 따른 OQ의 길이와 삼각형 넓이의 그래프를 함께 봄으로써, OQ가 최소일 때 넓이도 최소가 된다는 풀이의 핵심적인 아이디어를 훨씬 자연스럽게 받아들일 수 있었다.

특히 계산 결과를 검토하는 데도 유용했다. $Q(\frac{3}{2}, -\frac{1}{2})$ 일 때 넓이가 최소가 된다는 결론이 도형과 그래프 양쪽에서 동시에 확인되었기 때문에, 내가 구한 좌표와 최소 조건이 맞는지 검산하는 데 도움이 될 수 있었다. 다만 시뮬레이터는 어디까지나 직관과 검토를 돕는 도구였고, 왜 그 점에서 최소가 되는지를 미분을 이용하거나 식 변형으로 선분 OQ가 최소일 때임을 엄밀하게 증명하는 과정까지 대신해 주지는 못했다.

[두 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)

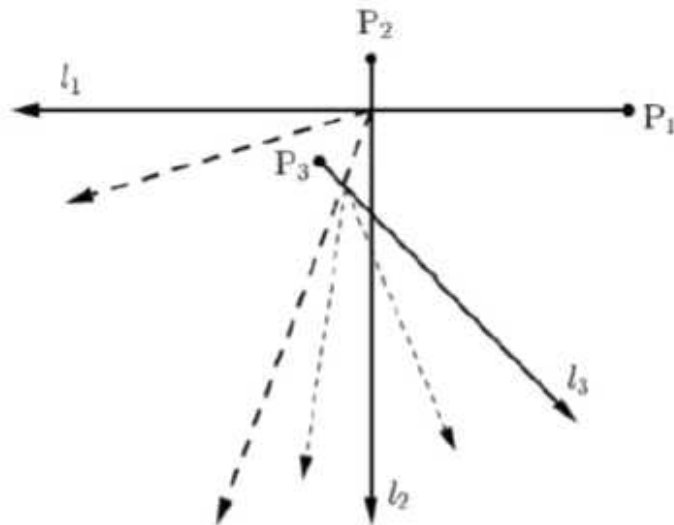
[문제 4] 좌표평면 위의 점 P 와 평면벡터 $\vec{v}$ 에 대하여, $\overrightarrow{OQ} = \overrightarrow{OP} + t\vec{v}$ ($t \geq 0$)을 만족하는 점 Q 가 그리는 반직선을 생각하자. 네 점 $P_1(6, 4)$, $P_2(1, 5)$, $P_3(0, 3)$, $P_4(0, -1)$ 과 네 벡터 $\vec{v}_1 = (-1, 0)$, $\vec{v}_2 = (0, -1)$, $\vec{v}_3 = (1, -1)$, $\vec{v}_4 = (1, 1)$ 에 대하여 $\overrightarrow{OQ_i} = \overrightarrow{OP_i} + t\vec{v_i}$ ($i = 1, 2, 3, 4$)를 만족시키는 점 Q_i 가 그리는 반직선 l_i 가 주어져 있다.

다음 제시문을 참고하여 물음에 답하시오.

[제시문] 두 개의 반직선이 만났을 때, 양의 실수 a 에 대하여 다음 규칙을 따라 새로운 반직선들이 생성된다고 하자.

- (가) 서로 다른 두 반직선 l_i, l_j ($1 \leq i, j \leq 4$)가 시작점 P_i 또는 P_j 가 아닌 점 P 에서 만날 때, $\overrightarrow{OR} = \overrightarrow{OP} + t(a\vec{v_i} + \vec{v_j})$ ($t \geq 0$)을 만족시키는 점 R 이 그리는 반직선과 $\overrightarrow{OS} = \overrightarrow{OP} + t(\vec{v_i} + a\vec{v_j})$ ($t \geq 0$)을 만족시키는 점 S 가 그리는 반직선이 각각 생성된다.
- (나) 새로 생성된 반직선들을 포함하여 임의의 서로 다른 두 반직선이 시작점이 아닌 점에서 만날 때에도 (가)의 규칙이 계속 적용된다.

※ 아래는 위 과정을 통해 생성된 일부 반직선들을 나타낸 그림이다.

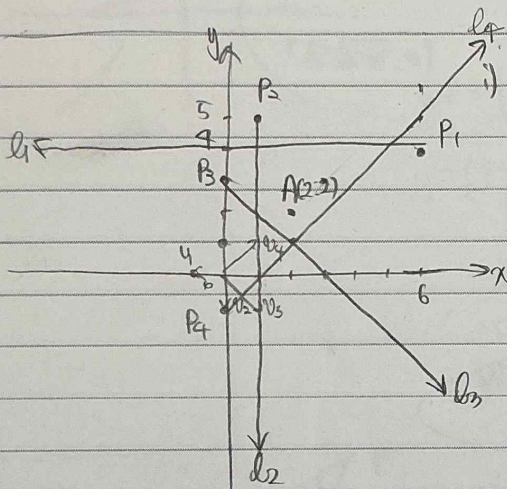


[문제 4-1] 두 반직선 l_1, l_2 에 대하여 [제시문]의 (가)에 따라 새로운 반직선들이 생성될 때, 점 $A(2, 2)$ 를 지나는 반직선이 생성될 수 있는 양의 실수 a 가 존재하면 그 값을 찾으시오. 만약 존재하지 않으면 그 이유를 설명하시오. [5점]

[문제 4-2] 네 반직선 l_1, l_2, l_3, l_4 에 대하여 [제시문]의 (가), (나)에 따라 새로운 반직선들이 생성될 때, 점 $A(2, 2)$ 를 지나는 반직선이 생성될 수 있는 양의 실수 a 가 존재하면 그 값을 찾으시오. 만약 존재하지 않으면 그 이유를 설명하시오. [10점]

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

규칙: P_i, P_j 가 만나면 l_i 와 l_j 의 선형결합을 단위벡터가 방향벡터이고, 시작이 P_i, P_j 의 교점인



1) 새 방향 $(1,4) + (0,-1) \rightarrow$ 법선벡터 $(1,0) \cdot (1,-2) \neq 0$
 or $(-1,0)$
 $\rightarrow$ 안맞음

같은 이끼로 (l_1, l_2)
 (l_1, l_3) $\rightarrow$ 안맞음
 (l_1, l_4)
 (l_2, l_3)
 (l_2, l_4)
 (l_3, l_4)
 (l_3, l_1)

㉠ 어떤 두 반직선의 교점이 의해 생성된 두 반직선 m_1, m_2 의 대해

$$m_1: \vec{OX}_1 = \vec{OP}_1 + t\vec{u}_1 \quad (\vec{u}_1 = (c_1, d_1) \rightarrow \text{수직벡터 } (-d_1, c_1) = \vec{w}_1)$$

$$m_2: \vec{OX}_2 = \vec{OP}_2 + t\vec{u}_2 \quad (\vec{u}_2 = (c_2, d_2) \rightarrow \text{수직벡터 } (-d_2, c_2) = \vec{w}_2)$$

$\rightarrow m_1, m_2$ 의 교점 P 를 지나는 어느 반직선

$$\vec{OP} = \vec{OP} + t\vec{v} \quad (t \geq 0) \quad (\vec{v} = a\vec{u}_1 + t\vec{u}_2 \text{ or } \vec{u}_1 + a\vec{u}_2)$$

$$\rightarrow \begin{cases} \vec{w}_1 \cdot \vec{PA} = \vec{w}_1 \cdot (\vec{PR}_1 + \vec{RA}) & \vec{w}_2 \cdot \vec{PA} = \vec{w}_2 \cdot \vec{RA} \neq 0 \\ = \vec{w}_1 \cdot \vec{RA} \neq 0 \end{cases}$$

$$1) \vec{v} = a\vec{u}_1 + t\vec{u}_2 \text{ 일 때 } \vec{v} = (ac_1 + tc_2, ad_1 + td_2)$$

$$2) \vec{v} = a\vec{u}_2 + t\vec{u}_1 \text{ 일 때}$$

$$\rightarrow \text{수직벡터 } \vec{w} = a\vec{w}_1 + t\vec{w}_2, \quad \vec{w}_1 \cdot \vec{PA} \neq 0, \quad \vec{w}_2 \cdot \vec{PA} \neq 0$$

$\therefore$ 새로 생성되는 모든 반직선은 점 A 안지남

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 벡터 문제는 몇 개의 반직선이 주어진 상태에서 끝나는 것이 아니라, 두 반직선이 만날 때마다 새로운 반직선이 계속 생성되는 재귀적인 과정을 다루고 있어서, 정적인 그림만으로는 전체 구조를 파악하기 어렵다고 느꼈다. 처음 주어진 네 반직선 l_1, l_2, l_3, l_4 의 시작점과 방향벡터만 보아서는, 어떤 교점에서 어떤 방향의 반직선이 새로 생기고 그 과정이 반복될 때 전체 도형이 어떻게 퍼져 나가는지 직관적으로 이해하기 쉽지 않았다. 따라서 시뮬레이션을 통해 반직선들이 실제로 어떻게 생성되고 확장되는지를 단계적으로 확인할 필요가 있다고 생각했다.

또한 이 문제는 단순히 그림을 보는 데서 끝나는 것이 아니라, 점 A(2, 2)를 지나는 반직선이 생성될 수 있는지의 여부를 판단해야해서 시뮬레이션이 문제 파악뿐 아니라 풀이에 대한 힌트를 얻는 데에도 중요하다고 느꼈다. 실제로 그림을 움직여보면 생성되는 반직선들이 점 A를 향해 가는 것처럼 보여도 결국 모두 점 A를 지나지 않는다는 사실을 시각적으로 확인할 수 있고, 이를 통해 “각 반직선에 대해 점 A가 항상 같은 쪽 반평면에 놓인다”는 아이디어를 떠올릴 수 있다. 즉, 시뮬레이션은 단순한 관찰 도구가 아니라, 해설에서 사용된 법선 벡터와 내적 부호 조건을 떠올리게 해 주는 역할을 할 수도 있다.

마지막으로, 시뮬레이션은 풀이를 검토하는 데에도 필요하다고 느꼈다. 해설에서는 새로 생성된 반직선의 방향벡터를 $\vec{A}$ 와 추가 벡터의 합 꼴로 나타내고, 그에 수직인 법선 벡터와 $\vec{PA}$ 의 내적이 항상 양수임을 반복적으로 보여 점 A를 지나는 반직선이 생성될 수 없음을 설명한다. 이 결론은 계산으로는 쉽게 보일 수 있지만, 실제 그림에

서도 모든 반직선이 점 A를 비껴간다는 사실이 함께 확인되어야 더 엄밀해진다. 따라서 시뮬레이터를 통해 생성 과정을 여러 단계까지 확인하면서, “양의 실수 a 가 존재하지 않는다”는 결론이 도형적으로도 맞는지 검산할 수 있다고 생각했다.

4. 사용한 AI 플랫폼을 모두 적으세요.

ChatGPT

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

다음 벡터 문제를 이해하고 풀이를 보조할 수 있는 HTML 기반 시뮬레이터를 만들어줘.

좌표평면 위에 네 점 $P_1 = (6, 4)$, $P_2 = (1, 5)$, $P_3 = (0, 3)$, $P_4 = (0, -1)$

과 네 방향벡터 $\vec{v}_1 = (-1, 0)$, $\vec{v}_2 = (0, -1)$, $\vec{v}_3 = (1, -1)$, $\vec{v}_4 = (1, 1)$ 가 주어져 있고, 각 점 P_i 를 시작점으로 하여 방향벡터 $\vec{v}_i$ 를 따라 나아가는 반직선 l_i 를 그려줘. 또한 점 $A = (2, 2)$ 도 함께 표시해 줘. 두 반직선이 시작점이 아닌 점에서 만나면, 양의 실수 a 에 대하여 그 교점 P에서 방향벡터가 $a\vec{v}_i + \vec{v}_j$ 또는 $\vec{v}_i + a\vec{v}_j$ 인 새로운 반직선이 생성되도록 구현해 줘. 이 생성 규칙은 새로 생긴 반직선들 사이에서도 계속 반복 적용되게 해줘.

시뮬레이터에는 다음 기능이 들어가야 해.

처음 주어진 네 반직선 l_1, l_2, l_3, l_4 를 서로 다른 색으로 표시하고, 시작점과 방향이 잘 보이게 해줘.

양의 실수 a 를 사용자가 슬라이더로 조절할 수 있게 해줘.

두 반직선의 교점이 생기면 그 점을 표시하고, 그 교점에서 생성되는 새로운 반직선들을 단계별로 그려줘.

“1단계 생성”, “2단계 생성”, “계속 생성” 같은 버튼을 두어서 생성 과정을 순차적으로 볼 수 있게 해줘.

점 $A(2, 2)$ 를 강조해서 표시하고, 어떤 반직선이 점 A를 지나면 즉시 따로 색을 바꾸거나 알림을 띄워줘.

각 반직선에 대해 방향벡터와 그에 수직인 법선 벡터를 함께 표시할 수 있는 옵션을 넣어줘.

사용자가 원하면 특정 반직선을 클릭해서 그 반직선의 시작점, 방향벡터, 그리고 점 A에 대해 법선 벡터와 $\overrightarrow{PA}$ 의 내적 부호를 확인할 수 있게 해줘.

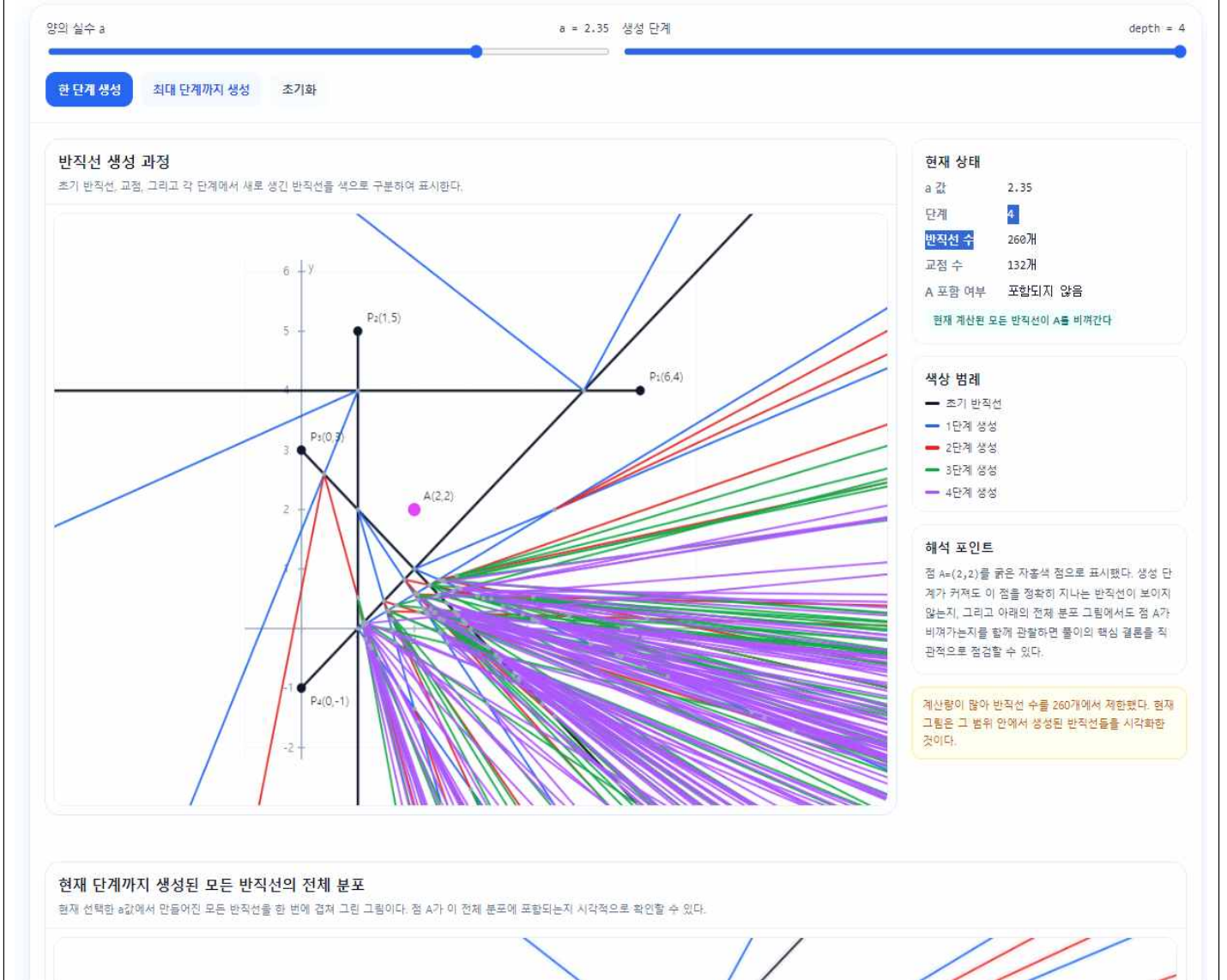
해설의 핵심이 모든 반직선이 점 A를 지나지 않는다는 거니까 점 A가 항상 각 반직선의 어느 쪽에 놓이는지 시각적으로 확인할 수 있도록 보조선이나 반평면 표시 기능도 넣어줘.

화면 오른쪽에는 현재 a 의 값, 생성된 반직선 개수, 점 A를 지난 반직선 존재 여부, 선택한 반직선의 방향벡터와 법선 벡터 정보를 표시해 줘. 맨 밑에는 모든 값에 대해서 만들어지는 모든 반직선이 차지하는 영역 그림도 그려 줘. 점 a 가 포함되는지, 안되는지 알기 위해서야. 외부 라이브러리 없이 하나의 HTML 파일에서 실행되게 하고, 보고서에 캡처해서 넣기 좋도록 깔끔한 화면 구성으로 만들어줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

반직선 생성 시뮬레이터

네 개의 초기 반직선 l_1, l_2, l_3, l_4 에서 출발하여, 교점이 생길 때마다 새 반직선을 생성하는 과정을 단계별로 시각화한다. 아래 맨 밑 그림에는 현재 선택한 a 값과 생성 단계까지의 모든 반직선을 한 번에 겹쳐 그려, 점 $A=(2,2)$ 가 포함되는지 직관적으로 확인할 수 있게 했다.



7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

맨 처음 시뮬레이터는 현재 선택한 하나의 a 값에 대해서만 반직선 생성 과정을 보여주었기 때문에, a 가 변할 때 생성되는 반직선들의 전체적인 분포를 확인하기 어려웠다. 따라서 정해진 생성 단계에서 버튼을 누르면 a 값이 자동으로 변하면서, 각 a 값에서 만들어지는 반직선들이 그래프 위에 누적되어 그려지도록 수정해야 했다. a 의 범위도 더 늘려야 했다.

또한 생성 단계가 낮으면 문제에서 요구하는 반복적인 반직선 생성 구조를 충분히 관찰하기 어렵기 때문에 생성 단계는 5단계까지 늘리고, 반직선 개수 제한도 더 확대했다. 마지막으로 맨 아래의 전체 분포 그림은 현재 a 값에서 생성된 반직선만 보여주는 것이 아니라, 전체 a 구간에 대해 생성될 수 있는 모든 반직선의 분포를 누적하여 색칠하도록 바꾸어야 했다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

1번째 수정: a 값은 버튼만 누르면 슬라이딩하면서 그려지게 해 주는 버튼도 만들어주고, depth를 한 5까지 늘려 줘. 반직선 수는 500개로 보고, 맨 밑의 그림은 현재 단계까지 생성된 모든 반직선의 전체 분포를 현재 a 값 말고 전체 a 에 대해서 색칠해 줘. 그래서 지나는 전체 영역을 색칠해서 안 지나는 부분 알 수 있게 해줘.

2번째 수정: 그러니까 결정된 생성 단계에서 a 값을 점진적으로 슬라이딩하면서 자동으로 그려지는 반직선들을 그

려줘. 지금은 버튼 눌러도 아무 일도 이루어지지 않아. 또 맨 밑에 전체 a 구간에 대한 반직선 분포 영역은 좀 더 축소 시켜서 더 많이 보이게 해줘. 한 1.5배 정도 줄여서.

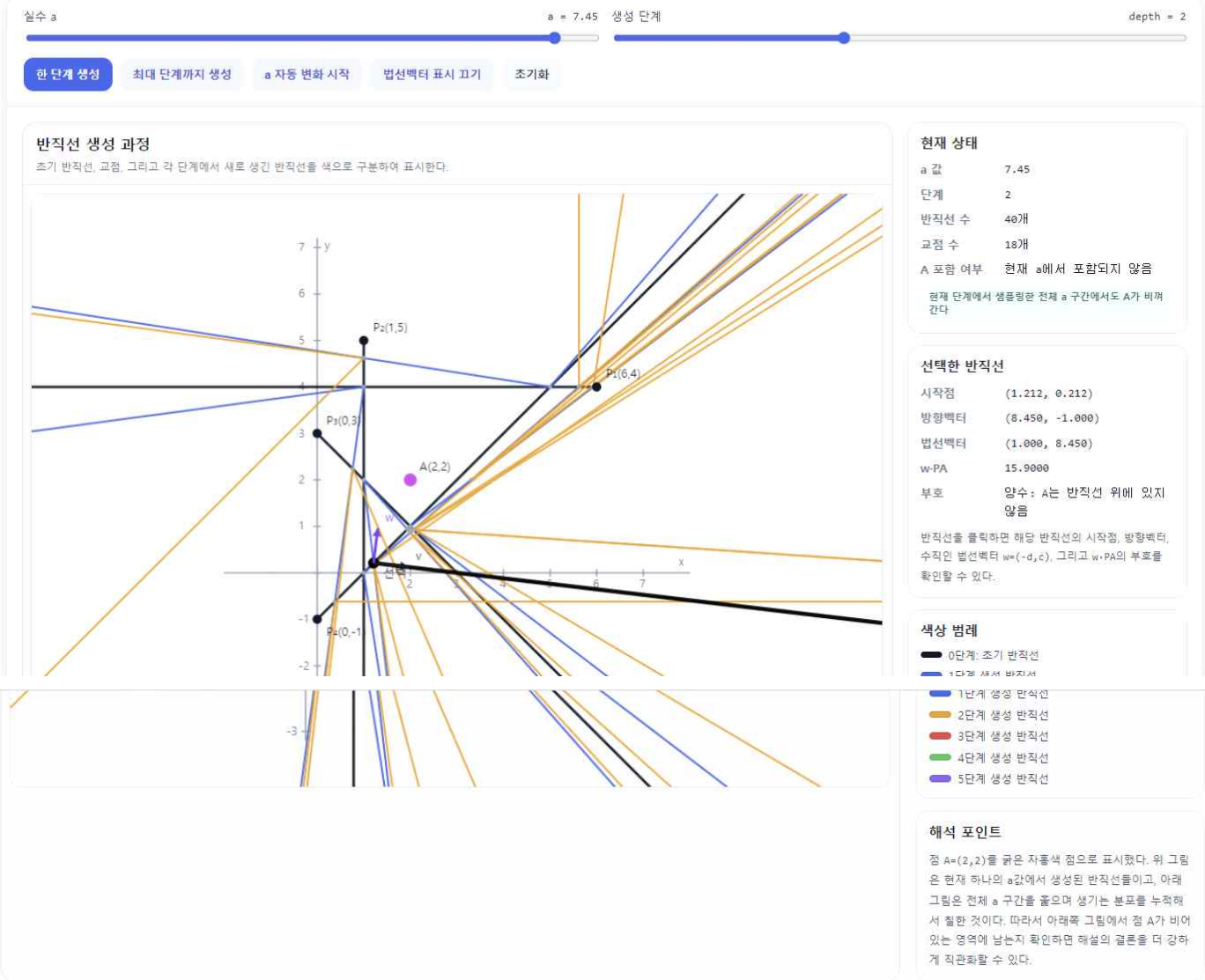
3번째 수정: 일단 a 값의 범위도 10까지만 늘려주고, 최소 반직선 수가 교점 수의 두 배 이상은 되어야 하니까 반직선 개수도 10000개로 해야할 것 같아. 반직선 생성 과정에 있는 큰 그래프 화면 아주 조금만 더 축소 시켜 줘. 1칸 정도? 색상 범례도 더 시인성 좋은거로 맞춰서 더 넣어주고, 설명을 낮은 단계, 높은 단계 이런 식으로 적지 말고 좀 더 구체적인 수치로 적어줘.

4번째 수정: 반직선 개수가 너무 많아서 렉이 걸려. 반직선 개수를 5000으로 줄이고 a 의 범위도 8까지로 해줘. 그리고 법선벡터 보이는 클릭한 직선, 다시 한 번 더 누르면 선택이 해제되게 해줘.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

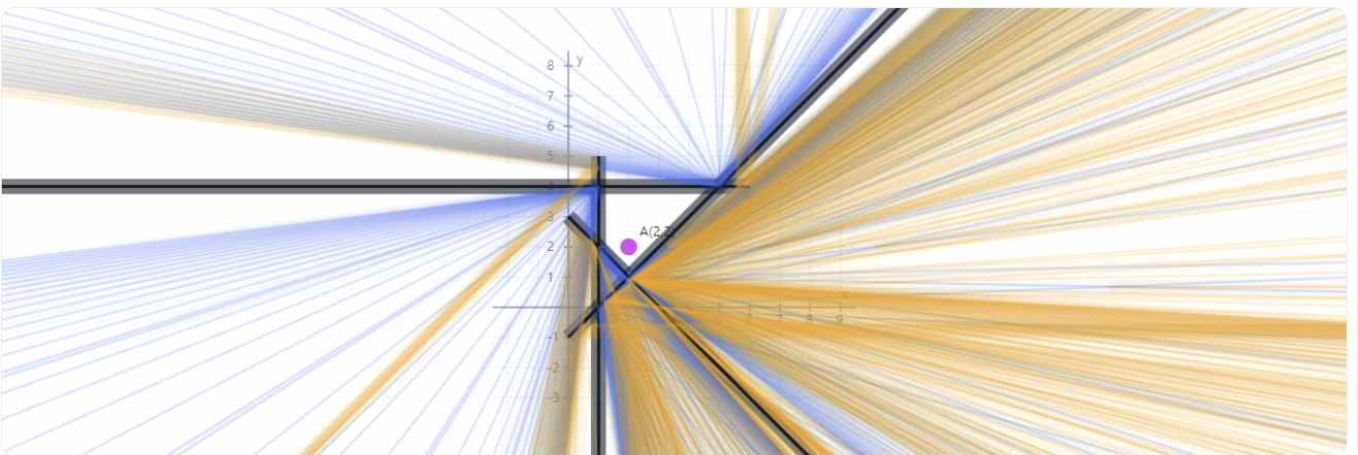
반직선 생성 시뮬레이터

네 개의 초기 반직선 l_1, l_2, l_3, l_4 에서 출발하여, 교점이 생길 때마다 새 반직선을 생성하는 과정을 단계별로 시각화한다. 아래 맨 밑 그림에는 현재 a 값이 아니라 전체 a 구간에 대해 현재 단계까지 생성될 수 있는 모든 반직선의 분포를 누적해서 색칠했다. 따라서 점 $A=(2,2)$ 가 전체적으로 포함되는지, 또는 끝까지 비껴가는지를 직관적으로 확인할 수 있다.



전체 a 구간에 대한 반직선 분포 영역

현재 단계까지, $a \in [0.00, 8.00]$ 전체 구간을 여러 값으로 층층히 샘플링하여 생성 가능한 모든 반직선의 분포를 색칠했다. 이 그림은 현재의 한 a 값과 무관하게, 전체적으로 지나가는 영역과 지나가지 않는 영역을 함께 보여준다.



10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

처음 실행하면 좌표평면 위에 네 개의 초기 반직선 l_1, l_2, l_3, l_4 와 점 $A(2, 2)$ 가 표시된다. 먼저 a 값 슬라이더를 움직여 양의 실수 a 를 바꾸면서 각 반직선의 방향이 새로운 생성 과정에 어떤 영향을 주는지 확인한다. 그 다음 1 단계 생성 또는 다음 단계 생성 버튼을 눌러, 기존 반직선들의 교점에서 새로운 반직선이 실제로 어떻게 만들어지는지 순서대로 관찰한다. 아니면 최대 단계로 생성을 원하면 최대 단계까지 생성 버튼을 눌러 5번째까지 만들어지는 반직선까지 관찰할 수 있다.

각 단계에서 새로 생긴 반직선은 색을 달리하여 표시하고, 특정 반직선을 클릭하면 그 반직선의 시작점, 방향벡터, 법선 벡터, 그리고 점 A 에 대한 위치 관계를 오른쪽 패널에서 볼 수 있게 한다. 만약 어떤 반직선이 점 A 를 지나면 화면에 즉시 표시되도록 하여 사용자가 조건 충족 여부를 바로 확인할 수 있게 한다. 이 과정을 통해 사용자는 점 A 를 지나는 반직선이 실제로 생성되는지 직접 확인할 수 있고, 생성이 반복되어도 여전히 점 A 를 지나지 않는다는 사실을 단계적으로 살펴볼 수 있다.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

이번 시뮬레이터는 특히 문제가 어떤 의미를 가지고 있는지에 대한, 어떻게 보면 문제 상황 분석 측면에서 매우 도움이 되었다. 이 문제는 처음 주어진 반직선 몇 개만 보고 끝나는 것이 아니라, 교점이 생길 때마다 새로운 반직선이 계속 생성되는 구조이기 때문에 직접 몇 개 그려보는 데는 한계가 있었다. 하지만 시뮬레이터에서는 어떤 교점에서 어떤 방향의 반직선이 얼마나, 어떻게 새로 생기는지를 (Depth) 단계적으로 볼 수 있어서, 문제의 재귀적 구조를 훨씬 분명하게 이해할 수 있었다.

또한 풀이에 대한 힌트를 얻는 데에도 도움이 되었다. 여러 값을 넣어 보고 생성 과정을 반복해서 관찰해도 반직선들이 점 $A(2, 2)$ 를 정확히 지나지 않는다는 사실이 유지되었고, 이를 보면서 단순히 개별 계산을 하는 것이 아니라 “점 A 가 모든 반직선의 한쪽에 놓인다”는 성질이 핵심이라는 점을 떠올릴 수 있었다. 이 시각적 관찰이 해설의 핵심인 법선 벡터 설정과 내적의 부호 판정으로 자연스럽게 이어졌다는 점에서 의미가 있었다. 대학 측이 발표한 해설에서도 각 반직선의 방향벡터에 수직인 벡터를 잡아 $\vec{w} \cdot \vec{PA} > 0$ 임을 보이고, 이 성질이 새로 생성된 반직선들에도 유지된다고 설명하고 있다.

마지막으로 이 시뮬레이터는 문제를 다 풀고 나서 결과에 대한 검토에도 도움이 되었다. 해설의 결론은 소문항 1번, 2번 모두에서 조건을 만족하는 양의 실수 a 가 존재하지 않는다는 것인데, 시뮬레이터를 통해 실제로 여러 a 값과 여러 생성 단계를 확인해도 점 A 를 지나는 반직선이 나타나지 않는다는 점을 시각적으로 검증할 수 있었다. 다만 a 값이 양의 실수 전체 값이 아니기 때문에 이 역시 한계가 존재했다.

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

기본적으로 컴퓨터 특성 상 계산의 능력은 인간에 비해서 매우 빠르다. 그렇기 때문에 복잡해서 우리의 손으로 직접 계산을 하지 못하는 분야는 컴퓨터의 도움이 절실하다. 하지만 수학 문제를 풀 때, 우리가 주도 해야 하는 분야들은 예를 들면 첫 번째 문제에서 넓이의 최소가 OQ의 최소와 연결된다는 판단, 점 Q의 x 좌표가 $\frac{3}{2}$ 에서 최솟값을 가진다는 결론을 얻기 위해선 우리가 직접 미분을 해보고, 그 값을 대입했을 때 실제로 만족하는지 확인하는 것들이다. 이런 것들은 단순히 AI에게 시키면 되지만 아직 오류가 많고, 수학 문제를 풀어서 실력을 함양하는 데 아무런 도움이 되지 않는다.

그렇지만, AI와 함께했을 때 공부의 효율이 극대화되는 부분은 바로 시각화이다. 예를 들어 두 번째 반직선 생성 문제에서는 교점에서 새로운 반직선이 반복적으로 생성되는 과정을 컴퓨터의 연산 능력으로 바로바로 시각화해 우리에게 직관 인사이트를 줄 수 있다. 또 생성된 반직선들이 점 A(2, 2)를 지나지 않는다는 사실을 시각적으로 관찰할 수는 있었지만, 이를 엄밀하게 증명하기 위해서는 우리가 직접 방향벡터에 수직인 법선벡터를 잡고 내적의 부호를 분석하는 논리적 사고가 필요했다. 이처럼 GPT는 코드 작성, 그래프를 구현하고, 변수값들을 변화 시켜가면서 문제 조건이 어떻게 변해가는지 시각화하는 데 효과적인 도구가 될 수 있지만, 조건을 해석하고, 풀이 방향을 결정하고, 수학적으로 해당 명제를 증명하는 일은 학생 본인이 책임져야 하는 영역이라고 생각한다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

복잡한 수학 문제를 AI와 함께 시뮬레이터로 구현해 본 경험은, 앞으로 공학적 문제를 해결할 때도 매우 중요한 역할을 할 수 있을 것 같다. 실제 공학적인 문제는 단순히 식 하나를 계산하는 방식으로 끝나지 않고, 여러 변수들이 서로 영향을 주며 변화하는 경우가 많다. 예를 들어 구조물의 하중 분포, 유체의 흐름, 필터 내부에서의 입자 이동, 기계 부품의 운동 등은 모두 시간이나 조건에 따라 계속 변화하는 동적 시스템인데, 우리 머릿속으로는 이게 구현해내기 어려울 때가 많다. 그렇기 때문에 이러한 문제를 해결하려면 먼저 수학적 모델을 세우고, 그 모델을 시뮬레이션으로 구현하여 실제 현상을 직관적으로 분석하는 과정이 필요하다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

이번 보고서를 작성하며 수학 문제를 푸는 과정이 단순히 계산을 정확히 하는 것에만 그치지 않는다는 것을 느꼈다. 시뮬레이터를 직접 바이브 코딩으로 만들어보고 문제점들을 수정하면서, 문제의 조건을 제대로 이해해야만 올바른 시각화가 가능하다는 것을 알게 되었다. 또한 그래프와 애니메이션을 통해 문제 푸는 데 직관을 얻을 수는 있지만, 최종적으로는 그 직관을 수식과 논리로 엄밀하게 설명해야 완벽한 풀이가 된다는 점도 다시금 느끼게 되었다. 특히 AI와 협업하는 과정에서 내가 원하는 기능을 명확히 이해할 수 있도록 설명하고, 결과를 검토하고, 필요한 부분을 계속 수정해 나가는 과정 자체가 어떻게 보면 또 하나의 문제 해결 과정이라고 느꼈다. 이번 활동을 통해 수학적 사고와 컴퓨터 시뮬레이션, 그리고 AI 활용 능력이 서로 유기적으로 잘 연결되면 큰 시너지를 만들어낼 수 있음을 경험할 수 있었다. 앞으로 실제 문제를 해결할 때도 AI를 단순한 답안 생성 도구로 사용하는 것이 아니라, 아이디어를 구현하고 실험하며 결과를 검토하고, 좀 더 사고를 확장해볼 수 있는 협업 도구로 활용해야겠다고 느꼈다.